

Node JS

Introduction to Node.js

Node.js is an **open-source, cross-platform JavaScript runtime environment** that allows developers to run JavaScript on the **server side**. It is built on the **Google V8 JavaScript engine**, which compiles JavaScript code into machine code for faster execution.

Installation and Setup of Node.js

Steps to Install Node.js

1. Visit the official website
<https://nodejs.org>
2. Download the **LTS version(Long Term Support)** of Node.js.
3. Run and install the setup file.
4. Verify installation using the terminal:

Check Node.js version:

```
node -v
```

Check npm version:

```
npm -v
```

- ✓ **Latest LTS version of Node.js is 24.x (LTS)** – recommended for production applications.
- ✓ **Latest Current version is 25.x** – includes the newest features but may change frequently.
- ✓ The Node.js installer includes the NPM(Node Package Manager).
- ✓ **NPM is the package manager** for the Node JS platform. It puts modules in place so that node can find them, and manages dependency conflicts intelligently.

Node.js Architecture

Node.js follows a **single-threaded, event-driven architecture** that allows it to handle multiple requests efficiently without creating multiple threads.

Key Components

1. **Client Requests**
Requests sent by users through a browser or application.
2. **Event Queue**
Stores incoming requests in a queue.
3. **Event Loop**
Continuously checks the queue and processes requests.
4. **Worker Threads**
Handle heavy operations such as file access, database queries, or network operations.

Example: Online Shopping Website

1. Client Requests

Users send requests to the server, for example:

- User A → View products
- User B → Add item to cart
- User C → Check order status

2. Event Queue

All incoming requests are placed in the **Event Queue**, which acts like a waiting line.

3. Event Loop

The **Event Loop** processes requests one by one from the queue.

4. Worker Threads

If a request requires heavy processing (such as database access or file reading), it is delegated to **worker threads**, while the event loop continues handling other requests.

Advantages

- Handles many users simultaneously
- Uses less memory
- Provides faster responses

Event Loop and Event-Driven Model

The **Event Loop** is the core mechanism that enables asynchronous operations in Node.js.

Event Loop Process

1. A client sends a request.
2. The request enters the **Event Queue**.
3. The **Event Loop** continuously checks the queue.
4. If the task is non-blocking, it is processed immediately.
5. If the task requires I/O operations, it is delegated to **worker threads**.
6. Once completed, the result is returned to the **Event Loop**, which sends the response to the client.

Event-Driven Model

Node.js operates on an **event-driven architecture**, where actions trigger events and corresponding **callback functions** are executed.

Environment Variables

What are Environment Variables?

Environment variables are **values stored outside the program** that are used to configure an application while it runs.

They are accessed in Node.js using:

```
process.env.VARIABLE_NAME
```

Why Do We Use Environment Variables?

1. **Security**

To store sensitive information like **passwords, API keys, tokens** instead of writing them in code.

2. **Different Environments**

The same program can run in **development, testing, and production** with different settings.

3. **Avoid Hardcoding**

Values like **port numbers or database URLs** should not be fixed inside the code.

When Do We Use Them?

Use environment variables when storing:

- Database credentials
- API keys
- Server port numbers
- Secret tokens
- Environment mode (development / production)

Step 1: Install dotenv

First install the package using npm:

```
npm install dotenv
```

This package allows Node.js to **read variables from the .env file**.

Step 2: Create a .env File

Inside your project folder, create a file named:

.env

Example:

```
PORT=3000
DB_PASSWORD=secret123
```

Step 3: Use dotenv in Your Code

```
require("dotenv").config();
console.log(process.env.PORT);
```

`require("dotenv")`

Imports the **dotenv module**.

`.config()`

Loads the variables from the `.env` file into **Node.js environment variables**.

`process.env.PORT`

Accesses the value of **PORT** stored in the `.env` file.

Asynchronous Programming in Node.js

Node.js supports **asynchronous programming**, meaning tasks can run without blocking the main execution thread.

Blocking vs Non-Blocking Operations in Node.js

In Node.js, operations are classified as **blocking** or **non-blocking** depending on whether the system waits for a task to complete before moving to the next task.

1. Blocking Operations

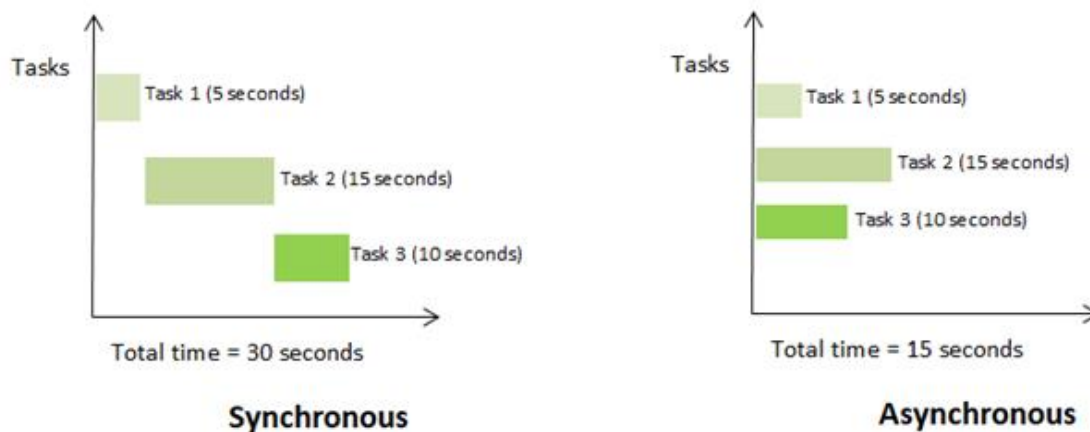
- A **blocking operation** stops the execution of the program until the current task is completed.
- When a request arrives, the system waits until the task finishes before processing another request.
- Reading a file from disk in a blocking way. If a user requests a large file, the server waits until the file is completely read before handling any other request.

2. Non-Blocking Operations

- A **non-blocking operation** allows the program to continue executing other tasks while waiting for the current task to complete.
- The system starts a task and immediately moves to the next request without waiting for completion.
- Reading a file asynchronously. User A requests a file. The server starts reading the file and immediately processes User B's request. When the file reading is completed, the response is sent to User A.

Asynchronous Programming Using Callbacks

The illustration below compares the time taken for tasks in both synchronous and asynchronous modes.



Callback

- ✓ A callback is a function that is passed as an argument to another function, and is called after the main function has finished its execution.
- ✓ The main function is called with a callback function as its argument, and when the main function is finished, it calls the callback function to provide a result.
- ✓ Callbacks allow you to handle the results of an asynchronous operation in a non-blocking manner, which means that the program can continue to run while the operation is being executed.
- ✓ **Callbacks allow you to continue executing code while the operation is being executed in the background. Once the operation has completed, the callback function is called with the result of the operation. This way, you can ensure that the program remains responsive and the user experience is not impacted.**

JavaScript setInterval() Method: The setInterval() method repeats a given function at every given time interval.

JavaScript setTimeout() Method: This method executes a function, after waiting a specified number of milliseconds.

How to create a Callback?

```
function main(cb) {
  console.log("Perform operation 1");
  // Use setTimeout to simulate an asynchronous operation
  setTimeout(function() {
    console.log("Operation completed");
  }, 2000);
  console.log("Perform operation 2");
}
// Define the callback_fun function
function callback_fun(result) {
  console.log("Result: " + result);
}
// Call the main function with the callback function
main(callback_fun);
```

Output:

```
Perform operation 1
Perform operation 2
Result: Operation completed
```

- ✓ We first define a **main** function that takes a **cb** as an parameter.

- ✓ The main function uses `setTimeout` to simulate an asynchronous operation. The `setTimeout` function takes two arguments: a function and a delay time in milliseconds.
- ✓ The `setTimeout` function calls the function with the result of the operation after the specified delay time.
- ✓ We then define a `callback_fun` that logs the result of the operation.
- ✓ Finally, we call the main function with the `callback_fun` as its argument.

We want to print a message in console but it should be printed after 3 seconds.

```
const message = function() {
  console.log("This message is shown after 3 seconds");
}
setTimeout(message, 3000);
```

- ✓ There is a built-in method in JavaScript called “`setTimeout`”, which calls a function or evaluates an expression after a given period of time (in milliseconds).
- ✓ So here, the “message” function is being called after 3 seconds have passed. (1 second = 1000 milliseconds)
- ✓ In other words, the message function is being called after something happened (after 3 seconds passed for this example), but not before. So the message function is an example of a callback function.

What is an Anonymous Function?

Alternatively, we can define a function directly inside another function, instead of calling it. It will look like this:

```
setTimeout(function() {
  console.log("This message is shown after 3 seconds");
}, 3000);
```

As we can see, the callback function here has no name and a function definition without a name in JavaScript is called as an “anonymous function”. This does exactly the same task as the example above.

Callback as an Arrow Function

If you prefer, you can also write the same callback function as an ES6 arrow function, which is a newer type of function in JavaScript:

```
setTimeout(() => {
  console.log("This message is shown after 3 seconds");
}, 3000);
```

Display content on browser after 5 seconds

```
<html>
```

```

<head>

</head>
<body>
  <p id="id"></p>
  <script>
    setTimeout(myfun,5000);
    function myfun()
    {
      document.getElementById("id").innerHTML="LJU";
    }
  </script>
</body>
</html>

```

Initialize two variables and increment both the variables each time and display the addition of both the variables at interval of 1 second.

```

<html>
<body>

<p id="p1"></p>

<script>
let a = 2, b = 5;

setInterval(() => {
document.getElementById("p1").innerHTML = ++a + ++b;
}, 1000);

</script>

</body>
</html>

```

Write code to increase the font size at interval of 50 ms and it should stop increasing when the font size reaches to 50px. This task should be performed when you click on "Increase button" on browser. (Default font size 15px)

```

<html>
  <body>

```



```

<p id="p1"> Hello</p>
<button onclick="fun1()">Increase</button>
<script>
font="15";
function fun(font)
{
    P1.style.fontSize=font;
    P1.style.color= "blue";
}
function fun1()
{
    setInterval(
        function()
        {
            if(font<=50)
            {
                fun(font++);
            }
        },50
    );
}
</script>
</body>
</html>

```

// Without using button

```

<html>
<body>
<p id="demo" style="color:blue"></p>
<script>
    size = 15;
    function add() {
        demo.innerHTML = "hello";
        demo.style.color ="red";
        demo.style.fontSize = size;
        if (size <= 50) {
            size++;
        }
    }
    setInterval(add, 1000);
</script>

```

```
</body>
</html>
```

Write code to perform the tasks as asked below.

- **Add three buttons.**
- **Increase button to increase the fonts. It should stop increasing the fonts when the font size reaches to 200px or stop button is clicked.**
- **Decrease button to decrease the fonts. It should stop decreasing the fonts when the font size reaches to 20px or stop button is clicked.**
- **Stop button to stop increasing or decreasing the fonts.**
- **Increasing/decreasing interval is of 100 ms.**
- **(Default font size = 50px)**

```
<html>
<body>
<p id="p1" style="font-size: 50px;"> Hello</p>
<button onclick="inc()">Increase</button>
<button onclick="dec()">Decrease</button>
<button onclick="stop()">stop</button>
<script>
font=50;
function fun(font){
document.getElementById('p1').style.fontSize=font+'px';
}
function inc()
{
test = setInterval(()=>{
if(font<100){ fun(++font);
}
},100);
}
function dec()
{
test =setInterval(()=>{
if(font>15){ fun(--font);}
},100);
}
function stop() { clearInterval(test); }
</script>
</body>
</html>
```


File System (fs) Module

- ✓ The 'fs' module of Node.js implements the File I/O operation.
- ✓ To include the File System module, use the require() method

The syntax for including the fs module in your application:

```
var fs=require("fs");
```

- ✓ Methods in the fs module can be synchronous as well as asynchronous.
- ✓ The Asynchronous function has a callback function as the last parameter which indicates the completion of the asynchronous function.
- ✓ Node.js developers prefer asynchronous methods over synchronous methods as asynchronous methods never block a program during its execution, whereas the latter does. We will discuss it later.

Operation	Async Method	Sync Method	Description
Read file	fs.readFile()	fs.readFileSync()	Reads the contents of a file
Write file	fs.writeFile()	fs.writeFileSync()	Creates or overwrites a file
Append file	fs.appendFile()	fs.appendFileSync()	Adds data to a file
Rename file	fs.rename()	fs.renameSync()	Renames a file or directory
Delete file	fs.unlink()	fs.unlinkSync()	Deletes a file
Create directory	fs.mkdir()	fs.mkdirSync()	Creates a directory
Remove directory	fs.rmdir()	fs.rmdirSync()	Removes a directory
Copy file	fs.copyFile()	fs.copyFileSync()	Copies a file

1. Read File

Async

```
fs.readFile(path, options, callback)
```

Sync

```
fs.readFileSync(path, options)
```

Example 1: Using toString()

```
const fs = require('fs');
// Async
fs.readFile('data.txt', (err, data) => {
  if (err) throw err;
  console.log(data.toString());
})
```

```
});
// Sync
const data = fs.readFileSync('data.txt');
console.log(data.toString());
```

- fs.readFile() returns **Buffer data**.
- data.toString() converts the buffer into **readable text**.

Example 2: Using "utf-8" Encoding

Instead of using toString(), we can specify encoding.

```
const fs = require('fs');
```

```
// Async
fs.readFile('data.txt', 'utf-8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

```
// Sync
const data = fs.readFileSync('data.txt', 'utf-8');
console.log(data);
```

Explanation

- "utf-8" tells Node.js to **return text instead of Buffer**.
- In this case, **toString()** is not needed.

2. Write File

Async

```
fs.writeFile(path, data, options, callback)
```

Example

```
const fs = require('fs');
// Async
fs.writeFile('data.txt', 'Hello Node.js', (err) => {
  if (err) throw err;
  console.log("File written successfully");
});
```

Sync

```
fs.writeFileSync(path, data, options)
```

Example

```
const fs = require('fs');
```

```
// Sync
```

```
fs.writeFileSync('data.txt', 'Hello Node.js');
console.log("File written successfully");
```

- Creates a new file.
- If the file already exists, it **overwrites the content**.

3. Append File

Async

```
fs.appendFile(path, data, callback)
```

Example

```
const fs = require('fs');
// Async
fs.appendFile('data.txt', '\nNew line added', (err) => {
  if (err) throw err;
  console.log("Content appended");
});
```

Sync

```
fs.appendFileSync(path, data)
```

Example

```
const fs = require('fs');
// Sync
fs.appendFileSync('data.txt', '\nNew line added');
console.log("Content appended");
```

- Adds new content to an **existing file**.
- Does **not overwrite** the file.

4. Rename File

Async

```
fs.rename(oldPath, newPath, callback)
```

Example

```
const fs = require('fs');
// Async
fs.rename('old.txt', 'new.txt', (err) => {
  if (err) throw err;
  console.log("File renamed");
});
```

Sync

```
fs.renameSync(oldPath, newPath)
```

Example

```
const fs = require('fs');
// Sync
fs.renameSync('old.txt', 'new.txt');
console.log("File renamed");
```

- Changes the **name of a file or directory**.

5. Delete File**Async**

```
fs.unlink(path, callback)
```

Example

```
const fs = require('fs');
// Async
fs.unlink('data.txt', (err) => {
  if (err) throw err;
  console.log("File deleted");
});
```

Sync

```
fs.unlinkSync(path)
```

Example

```
const fs = require('fs');
// Sync
fs.unlinkSync('data.txt');
console.log("File deleted");
```

- Deletes a file from the system.

6. Create Directory**Async**

```
fs.mkdir(path, options, callback)
```

Example

```
const fs = require('fs');
// Async
fs.mkdir('myFolder', (err) => {
```

```

    if (err) throw err;
    console.log("Folder created");
  });

```

Sync

```
fs.mkdirSync(path, options)
```

Example

```

const fs = require('fs');
// Sync
fs.mkdirSync('myFolder');
console.log("Folder created");

```

- Creates a **new directory (folder)**.

7. Remove Directory

Syntax

Async

```
fs.rmdir(path, callback)
```

Example

```

const fs = require('fs');
// Async
fs.rmdir('myFolder', (err) => {
  if (err) throw err;
  console.log("Folder removed");
});

```

Sync

```
fs.rmdirSync(path)
```

Example

```

const fs = require('fs');
// Sync
fs.rmdirSync('myFolder');
console.log("Folder removed");

```

- Deletes a directory.
- The folder must be **empty**.

8. Copy File

Async

```
fs.copyFile(src, dest, callback)
```

Example


```
const fs = require('fs');
// Async
fs.copyFile('source.txt', 'destination.txt', (err) => {
  if (err) throw err;
  console.log("File copied");
});
```

Sync

```
fs.copyFileSync(src, dest)
```

Example

```
const fs = require('fs');
// Sync
fs.copyFileSync('source.txt', 'destination.txt');
console.log("File copied");
```

- Copies a file from **source location** to **destination location**.

Note: We expect that the `fs.readFile()` would get the data in text format, i.e., string data type but it returns a buffer object.

In Node.js, the `Buffer.toString()` method is used to decode or convert a buffer to a string, according to the specified character encoding type. Converting a buffer to a string is known as encoding, and converting a string to a buffer is known as decoding.

UTF-8 is the World Wide Web's most common character encoding.

If no encoding is specified, then the raw buffer is returned.

Example to understand difference between sync and async.

```
var fs = require('fs');

//Synchronous
fs.writeFileSync('test.txt', 'Hello World!')
console.log('Synchronous Write operation completed.');
```

```
console.log("Outside");

//Asynchronous
fs.writeFile('test1.txt', 'Hello World!', function (err) {
```

```
if (err)
  console.log("Error Generated"+err);
else
  console.log(err)
  console.log('Asynchronous Write operation completed.');
```

});

console.log("Outside");

Output:

Synchronous Write operation completed.

Outside

Outside

Asynchronous Write operation completed.

Write node Example with File system sync methods. (CRUD Operation)

1. To create folder
2. Create one file inside that folder
3. Append some data to that file.
4. Read data from the file
5. Rename that file
6. Delete File

```
var fs=require("fs");
fs.mkdirSync("node");
fs.writeFileSync("node/write.txt","Hello");
fs.appendFileSync("node/write.txt","Hi");
data=fs.readFileSync("node/write.txt");
console.log(data);
console.log(data.toString()); //Or add "utf-8"

//data=fs.readFileSync("node/write.txt","utf-8");

fs.renameSync("node/write.txt"," node/readwrite.txt")
fs.unlinkSync("node/readwrite.txt");
```

Read data from file and sort that data in ascending order using .sort() .

//string format

```
var fs=require("fs");
fs.writeFileSync("s1.txt","50 -1 99 100 20 0 56 78 59");
data=fs.readFileSync("S1.txt","utf-8");
data=data.split(" ");
d = data.sort((a,b)=>a-b);
console.log(d);
```

Output:

```
[
  '-1', '0', '20',
  '50', '56', '59',
  '78', '99', '100'
]
```

Write a node.js script to copy contents of one file to another file. Data should be fetched from Source.txt and insert to destination.txt. And read data from destination file in the end. Perform all these tasks asynchronously.

```
const fs = require('fs');

// Step 1: Write data to Source.txt
fs.writeFile('Source.txt', 'ABC', (err) => {
  if (err) throw err;
  console.log("Source file created.");

  // Step 2: Read data from Source.txt
  fs.readFile('Source.txt', 'utf-8', (err, data) => {
    if (err) throw err;
    console.log("Data read from source:", data);

    // Step 3: Write data to destination.txt
    fs.writeFile('destination.txt', data, (err) => {
      if (err) throw err;
      console.log("Data copied to destination.txt");

      // Verify by reading destination file
      fs.readFile('destination.txt', 'utf-8', (err, data1) => {
        if (err) throw err;
        console.log("Destination file content:", data1);
      });
    });
  });
});
```

Using copyFile method

```
const fs = require('fs');

// Step 1: Create source file
fs.writeFile('Source.txt', 'This is source file data.', (err) => {
  if (err) throw err;
  console.log("Source file created.");

  // Step 2: Copy file
  fs.copyFile('Source.txt', 'destination.txt', (err) => {
    if (err) throw err;
    console.log("File copied successfully.");
  });
});
```

// Step 3: Read destination file

```
fs.readFile('destination.txt', 'utf-8', (err, data) => {  
  if (err) throw err;  
  console.log("Destination File Content:");  
  console.log(data);  
});  
});  
});
```

Write a node js script to write the text “You are creating a file” to help.txt file. After that append the text “you are appending data” to same help.txt file. After that read the file and print file contents on console. After finishing read operation , print the line “Thanks for using my program” on console. write ,append,read sequence must be maintain. all read ,write and append operations are asynchronous.

```
const fs = require('fs');
```

// Step 1: Write text to help.txt

```
fs.writeFile('help.txt', 'You are creating a file\n', (err) => {  
  if (err) throw err;  
  console.log("File created and data written.");
```

// Step 2: Append text to help.txt

```
fs.appendFile('help.txt', 'you are appending data\n', (err) => {  
  if (err) throw err;  
  console.log("Data appended.");
```

// Step 3: Read the file

```
fs.readFile('help.txt', 'utf-8', (err, data) => {  
  if (err) throw err;
```

```
  console.log("\nFile Contents:");  
  console.log(data);
```

// Step 4: Final message

```
  console.log("Thanks for using my program");
```

```
});
```

```
});
```

```
});
```

Write a Node.js script that asynchronously writes data to a file named 'test1.js'.

If no error occurs during the writing process, the script should then append additional data to the same file.

Finally, it should read the content of the file, including the newly written and appended data, and display it in the console.

Or

Writing data to file, appending data to file and then reading the file data using using ES6 callback.

```
fs = require("fs")
fs.writeFile('test1.txt', 'Hello World!', function (err) {
  if (err) { console.log("Error Generated"+err); }
  else{
    fs.appendFile('test1.txt', '\nGood Morning!', function (err) {
      if (err){ console.log("Error Generated"+err); }
      else{
        fs.readFile('test1.txt',"utf-8", (readErr, data) => {
          if (readErr) console.log("Error Generated: "+readErr)
          console.log(data);
        });
      }
    });
  }
});
```

OR Use setTimeout to read data

```
fs = require("fs")
fs.writeFile('test1.txt', 'Hello World!', function (err) {
  if (err) console.log("Error Generated"+err);
  fs.appendFile('test1.txt', '\nGood Morning!', function (err) {
    if (err)
      console.log("Error Generated"+err);
  });
});
```

```
var rfile = function(){  
  setTimeout(function() {  
    fs.readFile('test1.txt','utf-8', (readErr, data) => {  
      if (readErr) console.log("Error Generated: "+readErr)  
      console.log(data);  
    });  
  },2000)  
}
```

rfile();

JSON Processing Examples

Write node JS script to write an array of objects with properties name and age in a file named **student.txt**. Then read the file and display the object on console.

```
const student =
  [
    {
      name: "ABC",
      age: 30
    },
    {
      name: "XYZ",
      age: 32
    }
  ]
var ps=require("fs");

ps.writeFileSync("student.txt",JSON.stringify(student));
data=ps.readFileSync("student.txt","utf-8");

b=JSON.parse(data); // To access values of properties
console.log(b);
```

Output:

```
[ { name: 'ABC', age: 30 }, { name: 'XYZ', age: 32 } ]
```

Create JSON object which contains array of objects. Calculate perimeter of square and perimeter of circle by using side value and diameter value respectively. Write object as well as perimeter values of square and circle in shape.txt file.

```
const shape =
  [
    {
      name: "circle",
      diameter: 8
    },
    {
      name: "square",
      side: 10
    }
  ]
```

```
]
```

```
const shape =
[
  {
    name: "circle",
    diameter: 8
  },
  {
    name: "square",
    side: 10
  }
]
var ps=require("fs");
ps.writeFileSync("shape.txt",JSON.stringify(shape));
data=ps.readFileSync("shape.txt","utf-8");
b=JSON.parse(data);

var perimeter = (b[0].diameter/2) * 3.14 * 2 ; //2*3.14*r (here r=diameter/2)
console.log(perimeter);
var peri = (b[1].side) *4 ; //4*side
console.log(peri);

ps.appendFileSync("shape.txt","\nPerimeter of circle = "+ perimeter+ "\nPerimeter of square
= "+ peri);
```

Output:

```
[{"name":"circle","diameter":8},{"name":"square","side":10}]
```

```
Perimeter of circle = 25.12
```

```
Perimeter of square = 40
```

Write node js script and json to perform below tasks.

1. Write below object in txt file named **s2.txt**

```
{d:{a:10,b:20,c:[30,10]}}
```
2. Read data from the same file and perform the below tasks.
 - a. addition of a and b.
 - b. subtraction of 2nd element of c and b. (Must be positive value)
 - c. multiplication of elements of c.
3. Add the Output of addition, subtraction and multiplication below the object in s2.txt file.

```
var ps=require("fs");
const data ={d:{a:10,b:20,c:[30,10]}}

ps.writeFileSync("s2.txt",JSON.stringify(data)+"\n");
data1=ps.readFileSync("S2.txt","utf-8");
console.log(data1)
data1=JSON.parse(data1);
add=data1.d.a+data1.d.b;
sub=Math.abs(data1.d.c[1]-data1.d.b);
mul=data1.d.c[1]*data1.d.c[0];
console.log(add,sub,mul)

ps.appendFileSync("s2.txt",add+"\n"+sub+"\n"+mul)
```

Path Module

The Path module provides a way of working with directories and file paths.

The syntax for including the path module in your application:

```
var os=require("path");
```

Method	Description	Example
path.basename()	Returns the last portion of a path (file name)	path.basename('/home/user/file.txt')
path.dirname()	Returns the directory name of a path	path.dirname('/home/user/file.txt')
path.extname()	Returns the file extension	path.extname('file.txt')
path.parse()	Returns an object containing path information	path.parse('/home/file.txt')

Write node.js script to check whether the file extension is .txt or not.

```
var pm=require("path");
path=pm.dirname("D:/LJ/abc.html");
console.log(path);
path=pm.basename("D:/LJ/abc.txt");
console.log(path);
ext = pm.extname("D:/LJ/abc.txt")
console.log(ext);
path=pm.parse("D:/LJ/abc.html");
console.log(path);

if(path.ext == ".txt"){
  console.log("Text Document");
}else{
  console.log("Not a text Document");
}
```

Output:

```
D:/LJ
abc.txt
.txt
{
```

```

root: 'D:/',
dir: 'D:/LJ',
base: 'abc.html',
ext: '.html',
name: 'abc'
}

```

Not a text Document

Example

Write a **Node.js script** that performs the following operations using the **Path module** and the **asynchronous File System (fs) module**:

1. From a **given existing file path**, extract the **directory name** using the **Path module**.
2. **Create the extracted directory** inside an existing folder using the **File System module**.
3. Extract the **file name** from the given path using the **Path module**.
4. **Create that file** inside the newly created directory and write some data into it.
5. **Copy the content** of this file to another file.
6. **Delete the original file** after copying the content.

All file operations must be performed using **asynchronous methods of the fs module**.

```

const fs = require("fs");
const path = require("path");

// given existing path
let oldPath = "LJ/sample.txt";

// get directory from path
let dir = path.dirname(oldPath);

// get file name
let fileName = path.basename(oldPath);

// full file path
let newFilePath = dir + "/" + fileName;

/* Step 1: Create directory */
fs.mkdir(dir, (err) => {
  if (err) throw err;
  console.log("Directory created:", dir);
});

```

```
/* Step 2: Create original file */
fs.writeFile(newFilePath, "Sample Data for File", (err) => {
  if (err) throw err;
  console.log("Original file created:", newFilePath);

  /* Step 3: Copy file */
  fs.copyFile(newFilePath, dir + "/temp.txt", (err) => {
    if (err) throw err;
    console.log("File copied to temp.txt");

    /* Step 4: Delete original file */
    fs.unlink(newFilePath, (err) => {
      if (err) throw err;
      console.log("Original file deleted");

      console.log("All operations completed successfully");
    });
  });
});
});
```

Event Module

- ✓ In Node.js, an event emitter is like a messenger that helps different parts of a program communicate with each other. Imagine it as a broadcaster at a big event, letting everyone know what's happening.
- ✓ **Messenger** Role: Just like a messenger, an event emitter sends messages, or "events," to whoever is interested in hearing them. These events could be anything – a button click, a file being loaded, or a connection being established.
- ✓ **Listeners**: There are listeners, or the audience, who are interested in specific types of events. These listeners are set up to pay attention to certain messages. When the event emitter sends out an event, the relevant listeners respond to it.
- ✓ **Event** Types: Events have types, like categories. For instance, a 'click' event or a 'data received' event. When you're setting up an event emitter, you define these types, so others know what to expect.
- ✓ **Example**: Think of a light switch. The switch itself is like the event emitter. When you turn it on or off, it emits an event – the change in state. Now, imagine a smart home system with different devices, like lights and speakers. Each of these devices is a listener. They're interested in the 'state change' event. So, when the switch is toggled, these devices react accordingly.
- ✓ If you worked with JavaScript in the browser, you know how much of the interaction of the user is handled through events: mouse clicks, keyboard button presses, reacting to mouse movements, and so on.
- ✓ On the backend side, Node.js offers us the option to build a similar system using the events module.

You initialize that using

```
const EventEmitter = require('events');
const ee = new EventEmitter();
```

- ✓ **emit** is used to trigger an event
- ✓ **on/addListener** is used to add a callback function that's going to be executed when the event is triggered

Syntax:

```
eventEmitter.emit(event, [arg1], [arg2], [...])
eventEmitter.on(event, listener)
```

`eventEmitter.addListener(event, listener)`

- ✓ `eventEmitter.on(event, listener)` and `eventEmitter.addListener(event, listener)` are pretty much similar.
- ✓ It adds the listener at the end of the listener's array for the specified event. Multiple calls to the same event and listener will add the listener multiple times and correspondingly fire multiple times.
- ✓ Both functions return emitter, so calls can be chained.

In code, using an event emitter in Node.js looks something like this:

```
const EventEmitter = require('events');
const ee = new EventEmitter();
ee.on('start', () => {
  console.log('started');
});
ee.emit('start');
```

In this example, when the 'start' event is emitted, the listener responds by printing 'Started' to the console.

So, in Node.js, an event emitter is a handy tool for making different parts of a program talk to each other by sending and receiving messages. It's a way to create responsive and interactive applications.

Example with arguments

```
const EventEmitter = require('events');

const ee = new EventEmitter();
ee.on('start', (start, end) => {
  console.log(`started from ${start} to ${end}`);
});

ee.emit('start', 1, 100);
```

Output: started from 1 to 100

Example:

Write a Node.js program using the events module to simulate a sequence of events:

1. When a "connection" event occurs, print "Connection successfully" and trigger a "data-received" event.
2. When the "data-received" event occurs, print "Data received successfully".

3. Finally, print "Thanks" at the end of execution.

```
var EventEmitter=require("events");
var ee=new EventEmitter();

ee.on("connection",function(){
  console.log("Connection successfully");
  ee.emit("data-received");
});
ee.on("data-received",function()
{
  console.log("data received successfully");
})
ee.emit("connection");
console.log("thanks");
```

Output:

Connection successfully
data received successfully
thanks

Removing Listener:

The **eventEmitter.removeListener()** takes two argument event and listener, and removes that listener from the listeners array that is subscribed to that event. While **eventEmitter.removeAllListeners()** removes all the listener from the array which are subscribed to the mentioned event.

Syntax:

eventEmitter.removeListener(event, listener)

eventEmitter.removeAllListeners([event])

Note:

- Removing the listener from the array will change the sequence of the listener's array, hence it must be carefully used.
- The **eventEmitter.removeListener()** will remove at most one instance of the listener which is in front of the queue.

Example

Create a Node.js program using the events module to demonstrate:

1. Registering multiple event listeners for different events (myEvent1, myEvent2).
2. Removing a specific event listener (removeListener) for myEvent2.

3. Removing all listeners associated with myEvent1 (removeAllListeners).
4. Triggering events and observing which listeners execute.

//Importing events

```
const EventEmitter = require('events');
```

// Initializing event emitter instances

```
var eventEmitter = new EventEmitter();
```

```
var fun1 = (msg) => {
  console.log("Message from fun1: " + msg);
};
```

```
var fun2 = (msg) => {
  console.log("Message from fun2: " + msg);
};
```

// Registering fun1, fun2

```
eventEmitter.on('myEvent1', fun1); //executes fun1 function on event myEvent "Line A"
eventEmitter.on('myEvent2', fun2); //executes fun2 function on event myEvent "Line B"
eventEmitter.on('myEvent1', fun1); //executes fun1 function on event myEvent "Line C"
eventEmitter.on('myEvent2', fun2); //executes fun2 function on event myEvent2 "Line D"
```

// Removing a listener of myEvent2

```
eventEmitter.removeListener('myEvent2', fun2); //remove fun2 "line B"
```

// Removing all the listeners of myEvent1

```
eventEmitter.removeAllListeners('myEvent1'); // This will remove all events named "myEvent1"
Means (Line A) and (Line C)
```

// Triggering myEvent2

```
eventEmitter.emit('myEvent2', "LJ University"); //Executes a function fun2 (Line D)
eventEmitter.emit('myEvent1', "LJU"); // Nothing will be displayed as all events have been removed
```

Output:

```
Message from fun2: LJ University
```

Example

Write node js script to create two listeners for a common event. Print number of events associated with an emitter. Remove one of the listeners and call remaining listener again. Also, print number of remaining listeners.

```
const EventEmitter = require('events');
var ee = new EventEmitter();
var listener1 = function listen1() {
  console.log("Listener1 executed");
};
var listener2 = function listen2() {
  console.log("Listener2 executed");
};
ee.on("conn",listener1) // addListener/on both functions perform same task. You can use
addListener or on to execute a task.
ee.on("conn", listener2);

//Count initially all the events.
let count = ee.listenerCount("conn");
console.log("Count 1: " + count );
ee.emit("conn");

//To remove 1st listner
ee.removeListener('conn', listener1);
count = ee.listenerCount("conn");
console.log("Counting again: " + count );
ee.emit("conn");

// Above program ends here as per the question. Below is additional task of remove all
listeners. and count the listener.

//To remove all listners
ee.removeAllListeners('conn', listener1);
count = ee.listenerCount("conn");
console.log("Again Count afetr removing all listeners: " + count );
ee.emit("conn");
```

eventEmitter.listenerCount(): It returns the number of listeners listening to the specified event.

Write node js script to handle events as asked below.

- 1) Check the radius is negative or not. If negative then display message "Radius" must be positive" else calculate the perimeter of circle.
- 2) Check side is negative or not. If negative then display message "Side must be positive" else calculate the perimeter of square.

```
var EventEmitter = require("events");
var ee = new EventEmitter();

ee.on("negradius", () => {
  console.log("Radius must be positive");
});
ee.on("negside", () => {
  console.log("Side must be positive");
});
ee.on("findval", (r,s) => {

  if (r < 0) {
    ee.emit("negradius");
  }else{
    var rperi = 2 * 3.14 * r;
    console.log(rperi);
  }
  if (s < 0)
  {
    ee.emit("negside");
  }
  else{
    var speri = 4*s;
    console.log(speri);
  }
})
ee.emit("findval",10,3);
```

Write node js script to handle event to write data in file, append data in file and then read the file and display data in console.

```
var EventEmitter = require("events");
var fs = require("fs");
var ee = new EventEmitter();

/* Write Event */
ee.on("write", function () {
  fs.writeFile("b.txt", "Hello, ", function (err) {
    if (err) throw err;
    console.log("Data Written");
    ee.emit("append"); // trigger next event after write finishes
  });
});

/* Append Event */
ee.on("append", function () {
  fs.appendFile("b.txt", "Good Morning!", function (err) {
    if (err) throw err;
    console.log("Data Appended");
    ee.emit("read"); // trigger next event after append finishes
  });
});

/* Read Event */
ee.on("read", function () {
  fs.readFile("b.txt", "utf-8", function (err, data) {
    if (err) throw err;
    console.log("File Content:");
    console.log(data);
  });
});

/* Start Process */
ee.emit("write");
```

Write a node.js script using Event handling to perform following tasks in sequence:

- 1) Create a folder named abc.
- 2) Create a file in it named xyz.txt and enter data into it.
- 3) Append data to that file xyz.txt and print message "Data Appended Successfully".
- 4) Read the content of the file xyz.txt and print the content on console.
- 5) Copy content from xyz.txt to pqr.txt file.
- 6) Delete the file xyz.txt. And Lastly print the message "All operations performed successfully" on console.

Do all the operations of File using synchronous file system module.

```
var EventEmitter = require("events");
var fs = require("fs");
var ee = new EventEmitter();

/* Event 1: Create Folder */
ee.on("createFolder", function () {
  if (!fs.existsSync("abc")) {
    fs.mkdirSync("abc");
    console.log("Folder 'abc' created.");
  } else {
    console.log("Folder 'abc' already exists.");
  }
  ee.emit("createFile");
});

/* Event 2: Create File */
ee.on("createFile", function () {
  fs.writeFileSync("abc/xyz.txt", "This is the first line.\n");
  console.log("File xyz.txt created.");
  ee.emit("appendData");
});

/* Event 3: Append Data */
ee.on("appendData", function () {
  fs.appendFileSync("abc/xyz.txt", "This is appended data.\n");
  console.log("Data Appended Successfully");
  ee.emit("readFile");
});
```

/* Event 4: Read File */

```
ee.on("readFile", function () {  
  var data = fs.readFileSync("abc/xyz.txt", "utf-8");  
  console.log("File Content:");  
  console.log(data);  
  ee.emit("copyFile");  
});
```

/* Event 5: Copy File */

```
ee.on("copyFile", function () {  
  fs.copyFileSync("abc/xyz.txt", "abc/pqr.txt");  
  console.log("File copied to pqr.txt");  
  ee.emit("deleteFile");  
});
```

/* Event 6: Delete File */

```
ee.on("deleteFile", function () {  
  fs.unlinkSync("abc/xyz.txt");  
  console.log("xyz.txt deleted.");  
  console.log("All operations performed successfully");  
});
```

/* Start the process */

```
ee.emit("createFolder");
```